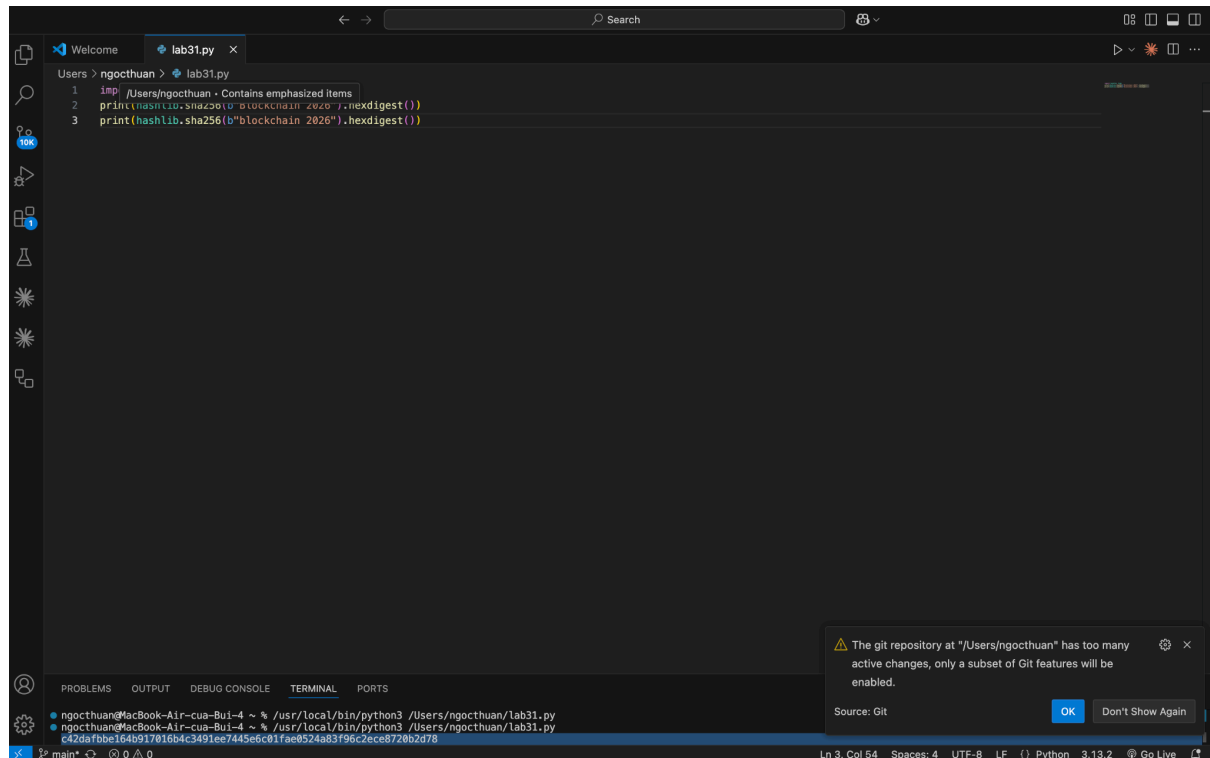
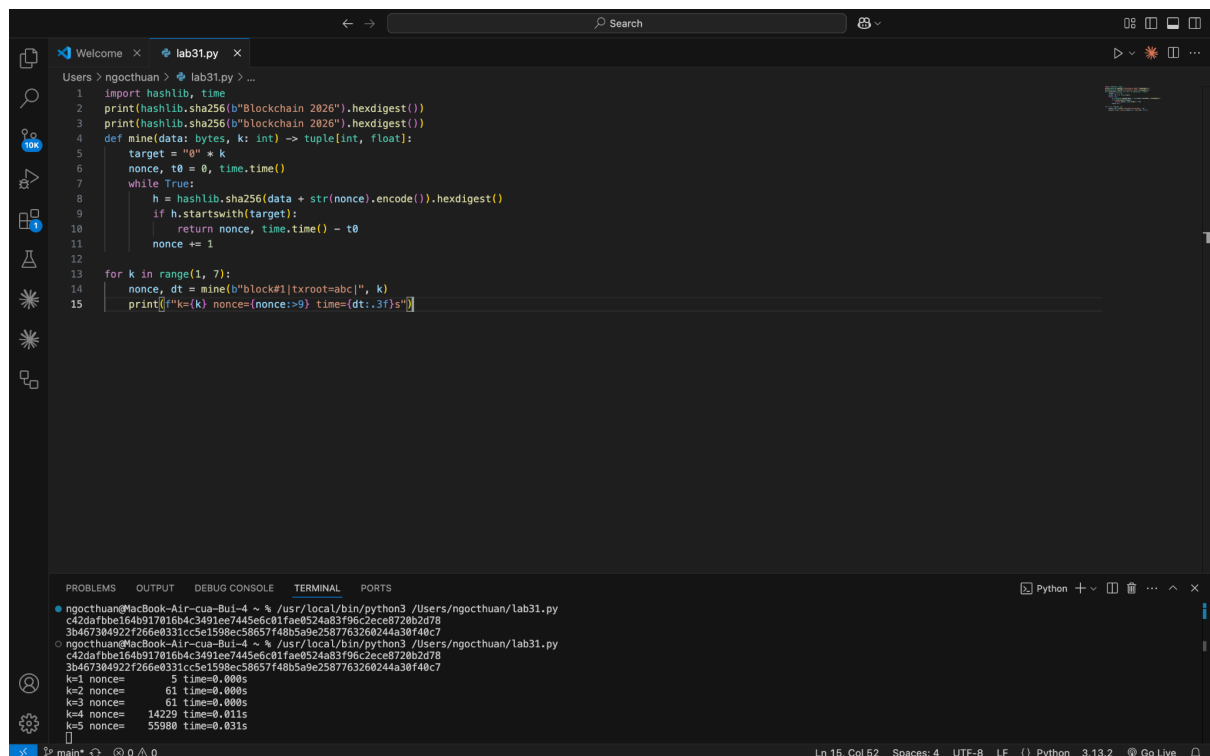


Lab 3.1



```
1 import hashlib
2 print(hashlib.sha256(b"blockchain 2026").hexdigest())
3 print(hashlib.sha256(b"blockchain 2026 ").hexdigest())
```

```
ngocthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 /Users/ngocthuan/lab31.py
c42dafbbe164b917016b4c3491ee7445e6c01fae0524a83f96c2ece8720b2d78
c42dafbbe164b917016b4c3491ee7445e6c01fae0524a83f96c2ece8720b2d78
```



```
1 import hashlib, time
2 print(hashlib.sha256(b"blockchain 2026").hexdigest())
3 print(hashlib.sha256(b"blockchain 2026 ").hexdigest())
4 def mine(data: bytes, k: int) -> tuple[int, float]:
5     target = "0" * k
6     nonce, t0 = 0, time.time()
7     while True:
8         h = hashlib.sha256(data + str(nonce).encode()).hexdigest()
9         if h.startswith(target):
10             return nonce, time.time() - t0
11         nonce += 1
12
13 for k in range(1, 7):
14     nonce, dt = mine(b"block#1|txroot=abc|", k)
15     print(f"k={k} nonce={nonce}>9} time={dt:.3f}s")
```

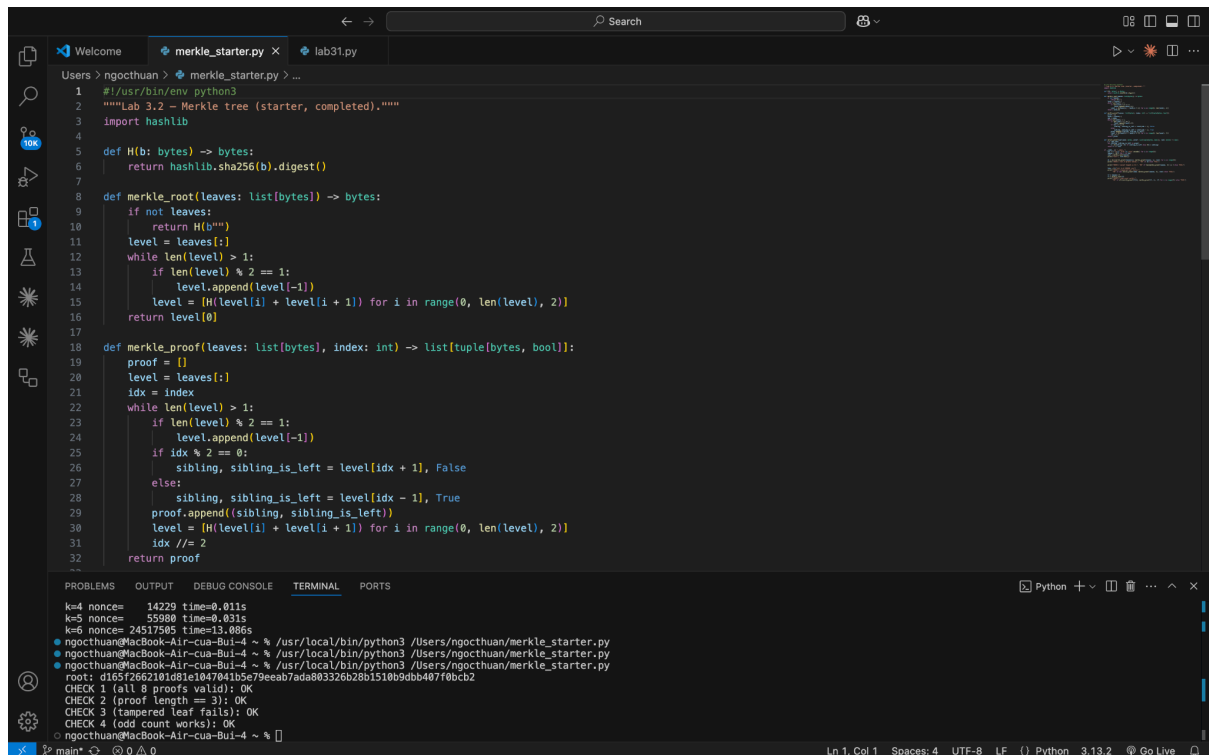
```
k=1 nonce= 5 time=0.000s
k=2 nonce= 61 time=0.000s
k=3 nonce= 61 time=0.000s
k=4 nonce= 14229 time=0.011s
k=5 nonce= 55980 time=0.031s
```

Q1: Theo lý thuyết, mỗi số 0 thêm vào target nhân kỳ vọng số lần thử lên $\times 16$, vì SHA-256 cho ra output phân bố đều trên bảng chữ hex 16 ký tự (0–f), xác suất 1 ký tự ngẫu nhiên là "0" luôn là $1/16$, không có cách nào đoán trước hay rút ngắn. Nhưng số liệu thực tế ($k=3$ đến 4 tăng $\times 233$, $k=5$ đến 6 tăng $\times 400$) không khớp đúng $\times 16$ vì mỗi lần chạy mine() chỉ là

một phép thử ngẫu nhiên duy nhất nên có phương sai lớn cho thấy rằng số liệu thực tế chỉ hội tụ về nhân 16 khi chạy rất nhiều lần và lấy trung bình.

Q2: Verify chỉ tốn 1 lần gọi hash duy nhất tính lại sha256(data+nonce) với nonce đã tìm được, so khớp với target. Điều này cho thấy PoW có tính bất đối xứng : tìm nonce thì tốn rất nhiều công sức (hàng triệu lần thử ở $k=6$), nhưng kiểm tra kết quả thì cực rẻ (1 phép tính). Đây chính là lý do PoW dùng được trong blockchain: ai cũng có thể verify nhanh mà không cần tin tưởng người đào.

Lab 3.2

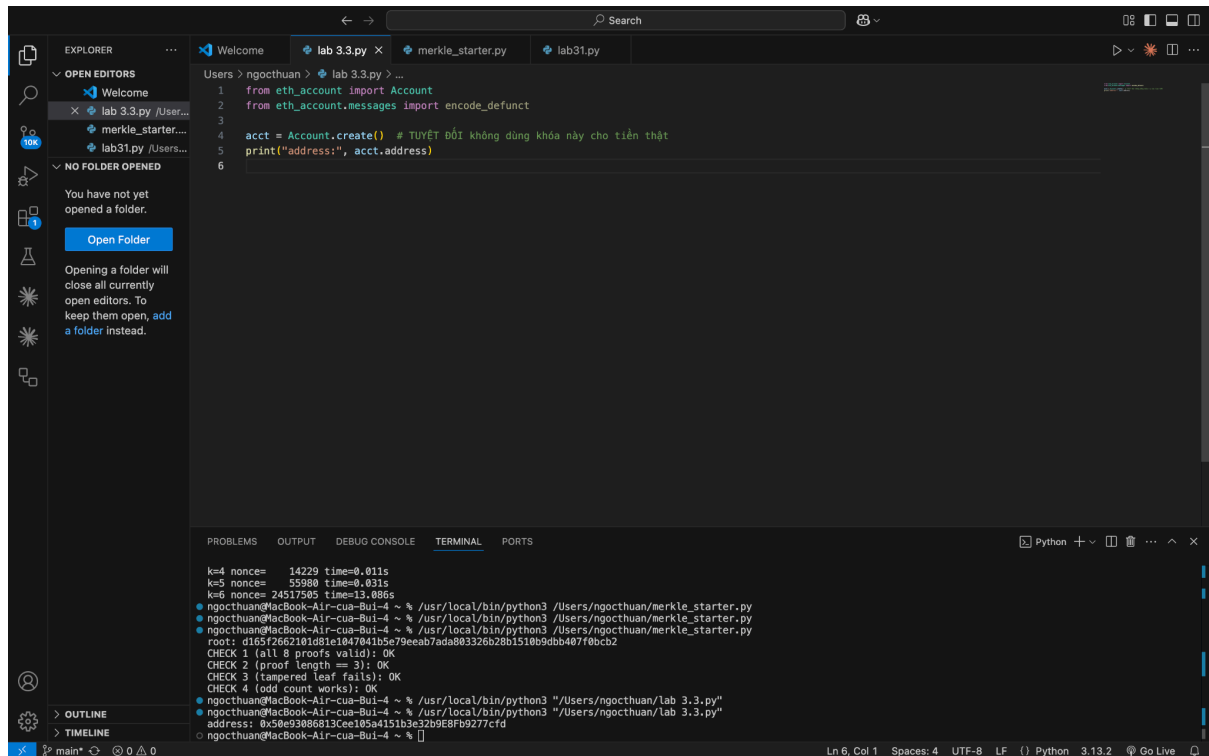


```
1 #!/usr/bin/env python3
2 """Lab 3.2 - Merkle tree (starter, completed)."""
3 import hashlib
4
5 def H(b: bytes) -> bytes:
6     return hashlib.sha256(b).digest()
7
8 def merkle_root(leaves: list[bytes]) -> bytes:
9     if not leaves:
10         return H(b'')
11     level = leaves[:]
12     while len(level) > 1:
13         if len(level) % 2 == 1:
14             level.append(level[-1])
15         level = [H(level[i] + level[i + 1]) for i in range(0, len(level), 2)]
16     return level[0]
17
18 def merkle_proof(leaves: list[bytes], index: int) -> list[tuple[bytes, bool]]:
19     proof = []
20     level = leaves[:]
21     idx = index
22     while len(level) > 1:
23         if len(level) % 2 == 1:
24             level.append(level[-1])
25         if idx % 2 == 0:
26             sibling, sibling_is_left = level[idx + 1], False
27         else:
28             sibling, sibling_is_left = level[idx - 1], True
29         proof.append((sibling, sibling_is_left))
30         level = [H(level[i] + level[i + 1]) for i in range(0, len(level), 2)]
31         idx //= 2
32     return proof
33
34 if __name__ == '__main__':
35     k=4 nonce= 14229 time=0.011s
36     k=5 nonce= 55980 time=0.031s
37     k=6 nonce= 24517505 time=13.086s
38     root: d165f2662101d81e1047041b5e79eeab7ada803326b28b1510b9dbb407f0bcb2
39     CHECK 1 (all 9 proofs valid): OK
40     CHECK 2 (proof length == 3): OK
41     CHECK 3 (tampered leaf fails): OK
42     CHECK 4 (odd count works): OK
43     o ngocthuat@MacBook-Air-cua-Bui-4 ~ %
```

Q3: Với $n = 1.000.000$ giao dịch, cây có độ sâu $\lceil \log_2(1.000.000) \rceil = 20$ tầng, nên một proof chứa 20 giá trị bằm thay vì phải tải cả triệu giao dịch, chỉ cần 20 hash là verify được 1 giao dịch có nằm trong cây hay không.

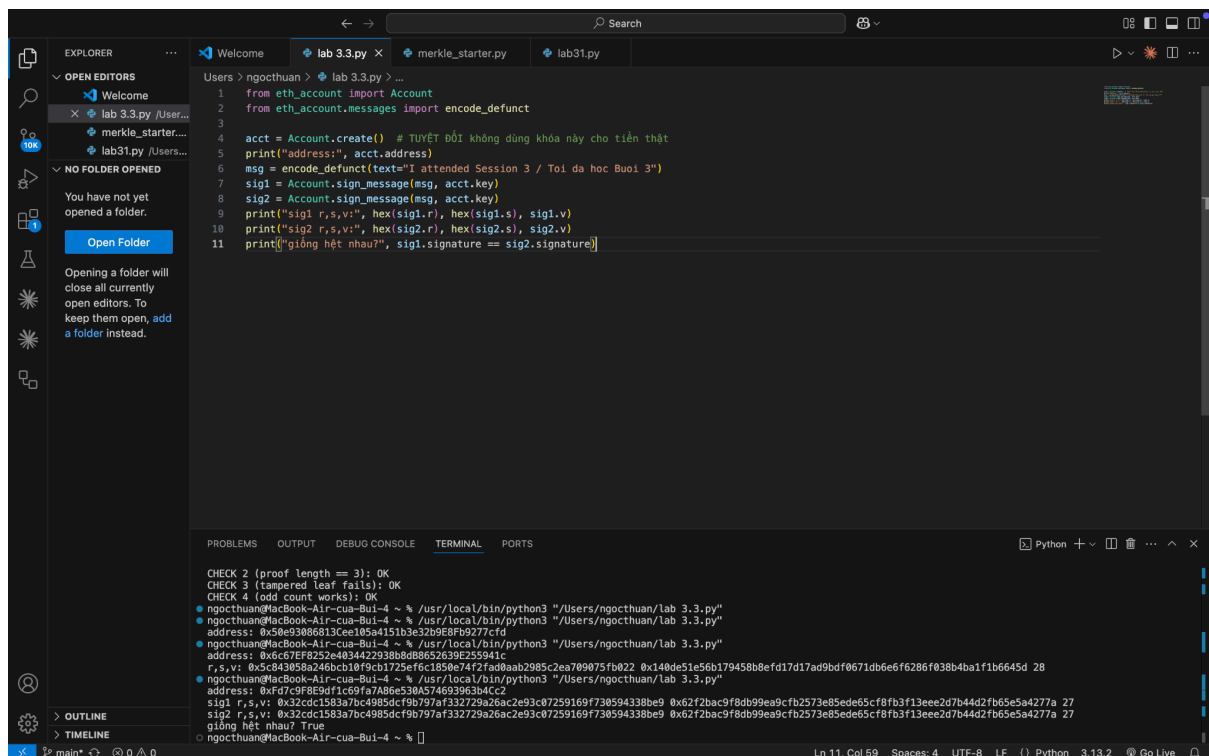
Q4: Ví dụ thực tế: SPV wallet (Simplified Payment Verification) trong Bitcoin ví nhẹ trên điện thoại không lưu toàn bộ blockchain, chỉ lưu block header (chứa Merkle root). Khi muốn biết 1 giao dịch có được xác nhận trong block hay không, ví chỉ cần full node gửi cho 1 Merkle proof (khoảng 20 hash), tự tính lại và so với root có sẵn trong header không cần tải cả block, không cần tin tưởng bên gửi proof.

Lab 3.3



```
Users > ngocthuan > lab 3.3.py > ...
1 from eth_account import Account
2 from eth_account.messages import encode_defunct
3
4 acct = Account.create() # TUYẾT ĐỐI không dùng khóa này cho tiền thật
5 print("address:", acct.address)
6
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
k=4 nonce= 14229 time=0.011s
k=5 nonce= 55980 time=0.031s
k=6 nonce= 24517505 time=13.086s
ngocthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 /Users/ngocthuan/merkle_starter.py
ngocthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 /Users/ngocthuan/merkle_starter.py
ngocthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 /Users/ngocthuan/merkle_starter.py
root: d165f266218d81e1847841b5e79eeab7ada803326b28b1510b9dbb487f0bc62
CHECK 1 (all 6 proofs valid): OK
CHECK 2 (proof length == 3): OK
CHECK 3 (tampered leaf fails): OK
CHECK 4 (odd count works): OK
ngocthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 "/Users/ngocthuan/lab 3.3.py"
ngocthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 "/Users/ngocthuan/lab 3.3.py"
address: 0x50e93886813Cee105a4151b3e32b9E8Fb9277cfd
ngocthuan@MacBook-Air-cua-Bui-4 ~ % []
```



```
Users > ngocthuan > lab 3.3.py > ...
1 from eth_account import Account
2 from eth_account.messages import encode_defunct
3
4 acct = Account.create() # TUYẾT ĐỐI không dùng khóa này cho tiền thật
5 print("address:", acct.address)
6 msg = encode_defunct(text="I attended Session 3 / Tôi đã học Buổi 3")
7 sig1 = Account.sign_message(msg, acct.key)
8 sig2 = Account.sign_message(msg, acct.key)
9 print("sig1 r,s,v:", hex(sig1.r), hex(sig1.s), sig1.v)
10 print("sig2 r,s,v:", hex(sig2.r), hex(sig2.s), sig2.v)
11 print("giống hệt nhau?", sig1.signature == sig2.signature)]
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
CHECK 2 (proof length == 3): OK
CHECK 3 (tampered leaf fails): OK
CHECK 4 (odd count works): OK
ngocthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 "/Users/ngocthuan/lab 3.3.py"
ngocthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 "/Users/ngocthuan/lab 3.3.py"
address: 0x50e93886813Cee105a4151b3e32b9E8Fb9277cfd
address: 0x6c67f8252c4034422938b8d88652639e255941c
r,s,v: 0x5c843058a246cb18f9cb1725ef6c1850e74f2fad0aab2985c2ea709075fb022 0x140de51e56b179458b8efd17d1ad9bdf0671db6e6f6286f038b4ba1f166645d 28
ngocthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 "/Users/ngocthuan/lab 3.3.py"
address: 0x6c67f8252c4034422938b8d88652639e255941c
sig1 r,s,v: 0x32cdc1583a7bc4985dcf9b797af332729a26ac2e93c07259169f730594338be9 0x62f2bac9f8db99ea9cfb2573e85ede65cf8fb3f13ee2d7b44d2fb65e5a4277a 27
sig2 r,s,v: 0x32cdc1583a7bc4985dcf9b797af332729a26ac2e93c07259169f730594338be9 0x62f2bac9f8db99ea9cfb2573e85ede65cf8fb3f13ee2d7b44d2fb65e5a4277a 27
giống hệt nhau? True
ngocthuan@MacBook-Air-cua-Bui-4 ~ % []
```

Task 1: Chữ ký giống hệt nhau vì eth-account dùng RFC 6979 nonce k trong ECDSA được sinh ra deterministic dựa trên hash của (private key + message), thay vì random như một số implementation cũ. Điều này tránh lỗi bảo mật nghiêm trọng: nếu k bị random yếu hoặc lặp lại giữa 2 chữ ký khác nhau, attacker có thể tính ngược ra private key (từng xảy ra thật với ví Bitcoin và máy PS3 của Sony).

```
1 from eth_account import Account
2 from eth_account.messages import encode_defunct
3
4 acct = Account.create() # TUYẾT ĐỒI không dùng khóa này cho tiền thật
5 print("address:", acct.address)
6 msg = encode_defunct(text="I attended Session 3 / Tôi đã học Buổi 3")
7 sig1 = Account.sign_message(msg, acct.key)
8 sig2 = Account.sign_message(msg, acct.key)
9 print("sig1 r,s,v:", hex(sig1.r), hex(sig1.s), sig1.v)
10 print("sig2 r,s,v:", hex(sig2.r), hex(sig2.s), sig2.v)
11 print("giống hệt nhau?", sig1.signature == sig2.signature)
12 who = Account.recover_message(msg, signature=sig1.signature)
13 print("recovered:", who, "| match:", who == acct.address)
14
15 bad = encode_defunct(text="I attended Session 3 / Tôi đã học Buổi 4")
16 who_bad = Account.recover_message(bad, signature=sig1.signature)
17 print("tampered ->", who_bad)
```

```
address: 0x6c67f8252e4034422938b8d88652639e255941c
r,s,v: 0x5c843058a246cb10f9cb1725ef6c1859e74f2fad08ab2985c26a789075fb022 0x148de51c56b179458b8cfd17d1ad9bdf0671db6e6f6286f038b4ba1f1b6645d 28
ngochthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 "/Users/ngochthuan/lab 3.3.py"
address: 0xf7c9f8e9df1c9fa7a86e538a574693963b4c2
sig1 r,s,v: 0x32cdc1583a7bc4985dcf9b797af332729a26ac2e93c07259169f730594338be9 0x62f2bac9f8db99ea9cfb2573e85ede65cf8fb3f13eee2d7b44d2f65e5a4277a 27
sig2 r,s,v: 0x32cdc1583a7bc4985dcf9b797af332729a26ac2e93c07259169f730594338be9 0x62f2bac9f8db99ea9cfb2573e85ede65cf8fb3f13eee2d7b44d2f65e5a4277a 27
giống hệt nhau? True
ngochthuan@MacBook-Air-cua-Bui-4 ~ % /usr/local/bin/python3 "/Users/ngochthuan/lab 3.3.py"
address: 0x8dd28e142f038c4526fb54c8593a2c200023c0a2
sig1 r,s,v: 0x40877e4d403839a5299a2c6a015e98f7c8983163ededaf177d5ebf997067f10 0x4b20ec08e16890c278ba9339f5f7ac2873d2e96b36e8ec4a3c34f58945fca477 27
sig2 r,s,v: 0x40877e4d403839a5299a2c6a015e98f7c8983163ededaf177d5ebf997067f10 0x4b20ec08e16890c278ba9339f5f7ac2873d2e96b36e8ec4a3c34f58945fca477 27
giống hệt nhau? True
recovered: 0x8dd28e142f038c4526fb54c8593a2c200023c0a2 | match: True
tampered -> 0x6870cct67079f2801820AF8e0fA038d771847C56
ngochthuan@MacBook-Air-cua-Bui-4 ~ %
```

Task 2 : Chữ ký ECDSA không được tính trực tiếp trên chuỗi text, mà trên hash của message. Chỉ cần đổi 1 ký tự, hash đã đổi hoàn toàn → phép toán recover_message giải ra một public key/address khác hẳn. Vì vậy nếu ai đó cố sửa nội dung message sau khi đã ký, địa chỉ khôi phục sẽ không khớp với người ký ban đầu. Đây chính là cách chữ ký số chứng minh tính toàn vẹn: chữ ký chỉ hợp lệ với đúng nội dung gốc, không hợp lệ với bất kỳ bản sửa nào.